SkillSphere Server Error Alerting — Merged Plan

Approach 1 (DIY Nodemailer) + Approach 3 (PM2 Hooks)

Date: 2026-10-24 **Status:** Thinking — Revised after peer review

---

Deployment Strategy — Phased Rollout

> ***Phase 1 first, Phase 2 only after validation.***

Phase	What Gets Deployed	When
Phase 1	`alertService.js` + crash hooks in `server.js` + controller `sendAlert()` calls	Now
Phase 2	`start.sh` bash wrapper + `pm2-crash-notify.js` + `ecosystem.config.js`	After a few days of Phase 1 running cleanly on real users

Why phased?

- Phase 1 is pure JS — zero risk to the running server, just a new service file and a few extra lines in catch blocks
- Phase 2 changes how PM2 starts the process (`pm2 reload` to new ecosystem config) — carries slightly more deployment risk
- If Phase 1 alerts work correctly and no false positives appear, Phase 2 is a safe and confident addition
- If Phase 1 reveals any issues (SMTP misconfigured, alert too noisy, duplicate firing), fix them before adding Phase 2 complexity

---

Why This Exists

Right now when something breaks on the server — like the OTP column being too small, a payment failing, or the process crashing — it sits silently in PM2 logs. You only find out when a user complains. This plan adds automatic email alerts so you know within minutes.

What Gets Built

Two complementary layers:

Layer 1 — Nodemailer Alerts (Approach 1)

Catches errors **inside** the Node.js process — both crashes and operational failures.

Layer 2 — PM2 Bash Wrapper (Approach 3)

Catches errors **at the process level** — when Node is killed by the OS (out of memory, SIGKILL) before it can run any code.

Together they cover every failure scenario.

What Triggers an Alert

Event	Layer	Throttled?
Server crash — unhandled throw	Layer 1 (`uncaughtException`)	Never
Unhandled promise rejection	Layer 1 (`unhandledRejection`)	Never
OTP save fails (DB error)	Layer 1 (controller catch)	1 per 15 min
Email not sending	Layer 1 (controller catch)	1 per 15 min
Payment failure	Layer 1 (controller catch)	1 per 15 min
Enrollment failure	Layer 1 (controller catch)	1 per 15 min
Process killed by OS (OOM/SIGKILL)	Layer 2 (`start.sh` exit hook)	File-lock 15 min
PM2 restart loop	Layer 2 (first restart immediate, then file-lock)	File-lock 15 min

> ***Change from v1:*** *Express global error handler is NOT hooked. Controllers already call* `sendAlert()` *directly — hooking the global handler too would send 2 emails for the same error (duplicate). The global handler remains for sending 500 responses only.*

> ***Change from v1:*** *PM2 restart alerts now use a* ***file-based lock*** *(writes timestamp to* `/tmp/ss-alert.lock` *) so a restart loop of 10 restarts sends 1 alert, not 10.*

Why 15-Minute Throttle?

Without throttling, the OTP bug seen in logs would have sent 50+ emails in 2 minutes — flooding your inbox until you start ignoring alerts. With 15-minute throttle:

- You get notified fast enough to act
- You don't get spammed
- Crashes and process exits use file-based throttle (works across process restarts)

What the Alert Email Looks Like

```
Subject: [SkillSphere ALERT] SequelizeDatabaseError — authController.js

Environment : production
Time        : 2026-10-23 19:17:41 UTC
Context     : POST /auth/login-otp

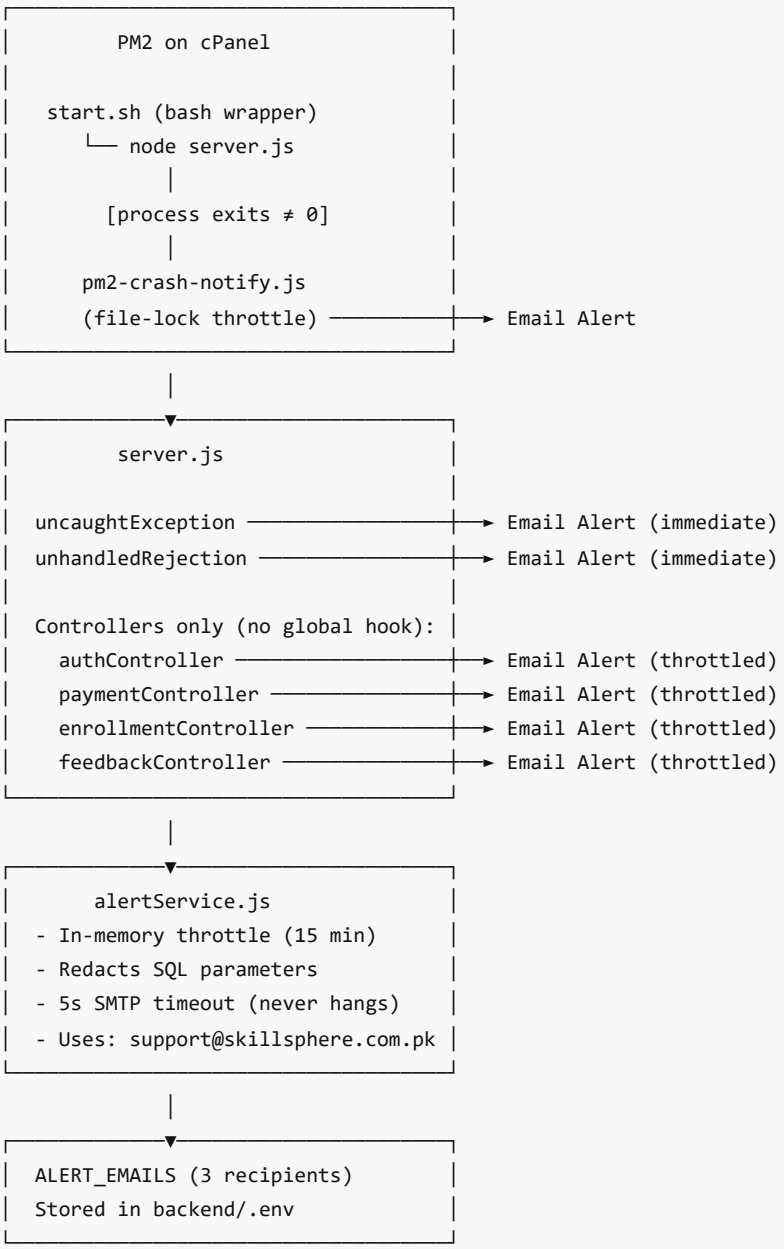
Error       : Data too long for column 'otpCode' at row 1

Stack Trace :
  at Query.run (sequelize/lib/dialects/mysql/query.js:52)
  at async exports.sendLoginOTP (authController.js:488)

SQL         : UPDATE users SET otpCode=?, otpExpiry=? WHERE id=?
              [SQL parameters redacted]
```

> ***Change from v1:*** *SQL parameters are* ***redacted*** *from emails. Parameters can contain OTP hashes, user IDs, or payment tokens. The SQL query itself is kept (useful for debugging) but values are replaced with* `[redacted]`.

How It Works — Architecture



Signal Handling in `start.sh`

A naive bash wrapper breaks `pm2 restart` because PM2 sends SIGTERM to bash, not to Node. The fix is a signal trap:

```
#!/bin/bash
_term() { kill -TERM "$NODE_PID" 2>/dev/null; }
trap _term SIGTERM SIGINT

node server.js &
NODE_PID=$!
wait $NODE_PID
EXIT_CODE=$?

# 0=clean  143=SIGTERM(pm2 restart)  130=SIGINT — all intentional, no alert
if [ $EXIT_CODE -ne 0 ] && [ $EXIT_CODE -ne 143 ] && [ $EXIT_CODE -ne 130 ]; then
  node pm2-crash-notify.js $EXIT_CODE 2>/dev/null &
fi
exit $EXIT_CODE
```

This means:

- `pm2 restart` → SIGTERM forwarded to Node → Node exits 143 → **no alert**
- `pm2 stop` → same, no alert
- Node OOM crash → exits non-zero → **alert fires**
- OS kills Node specifically → bash captures non-zero exit → **alert fires**

Files to Create or Change

New Files

File	Purpose
`backend/services/alertService.js`	Core: throttled `sendAlert()` and `sendCrashAlert()`, redacts SQL params, 5s SMTP timeout
`backend/pm2-crash-notify.js`	Standalone script called by `start.sh` — uses file-lock throttle
`backend/start.sh`	Bash wrapper with signal trap — runs Node, calls notify on non-zero exit
`backend/ecosystem.config.js`	PM2 config pointing to `start.sh` with `interpreter: bash`

Modified Files

File	Change
`backend/server.js`	Add `uncaughtException` + `unhandledRejection` hooks at top only — global error handler NOT hooked
`backend/controllers/authController.js`	Add `sendAlert()` to `sendLoginOTP`, `forgotPassword`, `login` catch blocks
`backend/controllers/paymentController.js`	Add `sendAlert()` to main catch blocks
`backend/controllers/enrollmentController.js`	Add `sendAlert()` to main catch blocks
`backend/controllers/feedbackController.js`	Add `sendAlert()` to main catch block

`backend/.env` (cPanel only — not committed)	Add `ALERT_EMAILS=email1,email2,email3`

Configuration

Only one thing needs to be added to `.env` on the server:

```
ALERT_EMAILS=youremail@gmail.com,secondemail@gmail.com,thirdemail@gmail.com
```

Everything else (SMTP host, port, user, password) is already configured.

No New Dependencies

Nodemailer is already installed. Zero new npm packages.

Deploy Steps

Phase 1 Deploy (Do This Now)

1. Upload to cPanel:
   - `backend/services/alertService.js` (new file)
   - `backend/server.js` (crash hooks added)
   - `backend/controllers/authController.js`
   - `backend/controllers/paymentController.js`
   - `backend/controllers/enrollmentController.js`
   - `backend/controllers/feedbackController.js`
2. Add `ALERT_EMAILS` to `.env` on the server:

   ```
   ALERT_EMAILS=youremail@gmail.com,secondemail@gmail.com,thirdemail@gmail.com
   ```

3. Restart server (simple restart, no PM2 config change):

   ```
   pm2 restart skillsphere --update-env
   ```

4. Verify still running:

   ```
   pm2 status
   pm2 logs skillsphere --lines 20
   ```

5. Test alert delivery — run on server:

   ```
   cd /home/skillsph/skill-sphere-Cpanel/backend
   node -e "require('./services/alertService').sendCrashAlert({ title: 'Test Alert',
   error: new Error('Phase 1 test'), context: 'manual test' })"
   ```

   Expected: email arrives within 30 seconds at all 3 addresses

6. Monitor for 2–3 days — watch for:
    - False positives (alerts for expected errors like 404s or validation failures)
    - Missing alerts (errors you see in logs but no email came)
    - SMTP issues

Phase 2 Deploy (After Phase 1 Validated)

Only proceed if Phase 1 has been running cleanly for a few days with no issues.

> ***Important:*** *Never use* `pm2 delete` *on production without a rollback plan. Use* `pm2 reload` *for zero-downtime switch.*

1. Upload to cPanel:
    - `backend/pm2-crash-notify.js` (new file)
    - `backend/start.sh` (new file)
    - `backend/ecosystem.config.js` (new file)
2. Make `start.sh` executable:

```
chmod +x /home/skillsph/skill-sphere-Cpanel/backend/start.sh
```

3. Switch PM2 to ecosystem config (zero-downtime):

```
pm2 reload /home/skillsph/skill-sphere-Cpanel/backend/ecosystem.config.js --env
production
pm2 save
```

4. Verify:

```
pm2 status
pm2 logs skillsphere --lines 20
```

5. Test PM2-level crash alert:

```
node /home/skillsph/skill-sphere-Cpanel/backend/pm2-crash-notify.js 999
```

    Expected: alert email arrives within 30 seconds

Rollback Plan

If PM2 fails to start after switching to ecosystem config:

```
pm2 delete skillsphere
pm2 start /home/skillsph/skill-sphere-Cpanel/backend/server.js --name skillsphere
pm2 save
```

This restores the original setup in under 30 seconds.

What Alerts Will NOT Cover

Scenario	Why	Mitigation
SMTP server down during crash	Can't send email without SMTP	Check cPanel mail server uptime separately
Entire server unreachable	No code can run	Add UptimeRobot (free) to ping `/api/health` every 5 min
Bash itself killed by SIGKILL	No code runs after SIGKILL to bash	Extremely rare; Layer 1 covers JS-level crashes

> ***Recommended future addition:*** *Register a free UptimeRobot monitor on* `https://skillsphere.com.pk/api/health` *. Takes 2 minutes and covers the scenario where the entire server goes down — no code changes needed.*

Changes Made After Peer Review

Issue Raised	Fix Applied
Duplicate alerts (global handler + controller)	Removed global Express error handler hook
PM2 restart loop flooding inbox	File-based lock in `pm2-crash-notify.js` (15 min window)
SQL parameters in emails exposing sensitive data	Parameters redacted, query kept
`pm2 delete` dangerous on production	Changed to `pm2 reload` with rollback plan
Bash wrapper signal handling	Signal trap added, exit codes 0/143/130 excluded from alerts

---

Summary

Question	Answer
New npm packages needed?	None
DB changes needed?	None
Cost?	Free
Covers OTP bug scenario?	Yes — Layer 1 controller catch
Covers OOM/SIGKILL?	Yes — Layer 2 bash wrapper
Covers crashes with no try/catch?	Yes — `uncaughtException` hook
Duplicate alerts possible?	No — global handler not hooked
Safe to deploy on live production?	Yes — `pm2 reload` is zero-downtime

Sensitive data in emails?	No — SQL parameters redacted